

Optimizing Breadth-First Search on the NVIDIA DGX Spark

Student: Fabio Missagia

Supervisors: Flavio Vella, Thomas Pasquali, Jacopo Raffi

Department of Information Engineering and Computer Science, University of Trento



UNIVERSITÀ
DI TRENTO

 HICREST

NVIDIA DGX Spark: general properties

Property	Value
System	NVIDIA DGX Spark
CPU	20 cores: 10× Cortex-X925 + 10× Cortex-A725 (aarch64)
GPU	NVIDIA GB10 — compute capability 12.1 (sm_121)
Memory	128 GB unified LPDDR5x, CPU–GPU coherent, ~ 273 GB/s <i>shared</i>
L2 cache	24 MiB
SMs	48
GPCs	4 (12 SMs each) — <i>measured, see next slides</i>
Max grid	$2\,147\,483\,647 \times 65\,535 \times 65\,535$

One *physical* memory pool, hardware coherent: a kernel dereferences a plain `malloc()` pointer with no `cudaMemcpy`. ~ 273 GB/s is an order of magnitude below datacenter HBM, like H100 (~ 2.0 TB/s).

NVIDIA DGX Spark: per-SM properties

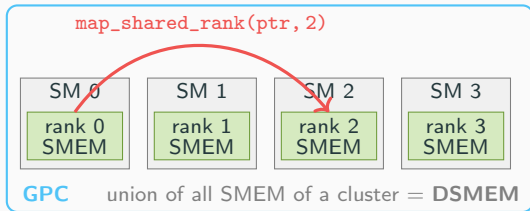
Property	Value
Max threads / block	1024
Max threads / SM	1536 (48 warps)
Max blocks / SM	24
Registers / SM	65 536
Unified L1 + shared memory	128 KiB
Max shared memory / SM	102 400 B (100 KiB)
Default shared memory / block	49 152 B (48 KiB)
Max shared memory / block (opt-in)	101 376 B (99 KiB)
Reserved shared memory / block	1 024 B

L1 and shared memory are the *same* 128 KiB SRAM, split by a configurable carveout: shared memory is bought at the cost of L1.

Thread block clusters (compute capability 9.0+)

An optional level between block and grid:

- **adjacent thread blocks** are grouped into clusters
- guaranteed to be **co-scheduled simultaneously inside one GPC**
- any thread block can address **any peer's shared memory** — loads, stores and *atomics* ride an **SM-to-SM** interconnect, not L2/DRAM $\Rightarrow$ **DSMEM**
- **co-residency** is guaranteed
- hardware barrier across blocks: `cluster.sync()`



Thread block clusters on the DGX Spark

Measurement	portable	with opt-in
Cluster Size	8	12

- **8** is the maximum CUDA guarantees *portably* on any cluster-capable GPU — code using it runs anywhere.
- **12** is *this hardware's* real maximum, unlocked per kernel with `cudaFuncAttributeNonPortableClusterSizeAllowed` — at the price of code that is no longer portable. Above 12: *cluster misconfiguration*.
- `cudaOccupancyMaxPotentialClusterSize` is **per kernel**, not a device constant: the same kernel answers 8 before the opt-in and 12 after.
- 12 is *usable*, not merely launchable: DSMEM verified across all 12 ranks with a ring test (rank r reads rank $r+1 \bmod n$).

Clusters are not free: TODO

The moment a launch carries a cluster dimension, the device holds **2 blocks per SM — 96 in total — whatever the block size**. Confirmed two independent ways: `cudaOccupancyMaxActiveClusters`, and a rendezvous test in which every block must observe all the others before a timeout.

blockDim	unclustered	clustered	penalty	occupancy
64	1152	96	12×	8%
128	576	96	6×	17%
256	288	96	3×	33%
512	144	96	1.5×	67%
1024	48	48	none	67%

- Every cluster size multiplies out to the same 96: sizes 1/2/4/6/8/12 give 96/48/24/16/12/8 clusters — as if there were *2 cluster slots per SM*.
- **Not** registers or warps (unclustered the same kernel gets 24 blocks/SM), **not** shared memory (it allocates none; the cap holds at every carveout), **not** the cluster API (free without a cluster dimension). The cap appears only when the launch carries one.
- **The lever is block size, not cluster size**: 512-thread blocks keep 67% occupancy where the natural-looking 128 silently loses 4×. Mechanism undocumented by NVIDIA — *still investigating*.

How many GPCs?

Problem. No CUDA attribute exposes the GPC count or the SM-to-GPC mapping.

Idea. Exploit the one guarantee available: *all blocks of a cluster are co-scheduled in a single GPC*. So two SMs seen hosting blocks of the *same* cluster are provably in the *same* GPC.

Method.

- every block reads the `%smid` register and reports the SM it ran on;
- **union-find** over the co-occurrences — the connected components *are* the GPCs;
- 300 trials, until all 48 SMs have been observed;
- 52 KB of shared memory per block forces one block per SM, so a 12-block cluster is guaranteed to cover 12 *distinct* SMs.

GPC topology: 4 GPCs $\times$ 12 SMs

GPC	SM ids											
GPC 0	0	1	8	9	16	17	24	25	32	33	40	41
GPC 1	2	3	10	11	18	19	26	27	34	35	42	43
GPC 2	4	5	12	13	20	21	28	29	36	37	44	45
GPC 3	6	7	14	15	22	23	30	31	38	39	46	47

Derived and verified:

- $\text{TPC} = \text{smid}/2$, $\text{GPC} = (\text{smid}/2) \% 4$
- 24 TPCs of 2 SMs, 6 TPCs per GPC, round-robin across GPCs.

Three independent cross-checks:

- union-find yields 4 components of 12 SMs each;
- $4 \times 12 = 48$ matches `multiProcessorCount`;
- the largest launchable cluster (12) *equals* one GPC's SM count

Latency Benchmark – Random pChase

Dependent random pointer chase, 16 384 steps, one thread, median of 101 reps.

- **Sattolo cycle.** One random cycle over all elements: no short cycles, nothing for a prefetcher.
- **Dependency defeats overlap.** Load $i+1$ needs the value of load i : pure latency, no MLP.

Placement	GB10 (cy)	ns	H800 (cy)*
SMEM	32.9	13.7	29.0
SMEM, via Cooperative Groups API	36.9	15.3	33.0
L1 Cache	42.9	17.9	32.0
L2 Cache	340.1	141.7	264–502
DRAM	965.8	401.6	656

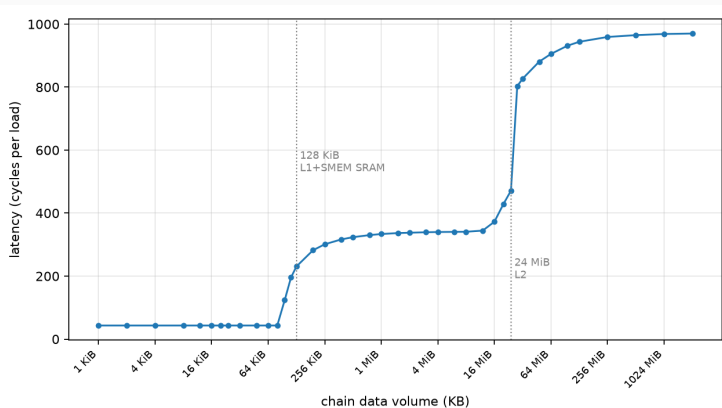
* Luo et al., arXiv:2501.12084 — Hopper H800, same p-chase method

Why does SMEM via DSMEM cost 4 more cycles? `map_shared_rank` returns a generic pointer, so the load compiles to LD (address space resolved by hardware on every access) instead of LDS (a plain shared-memory load).

Latency Benchmark – Random pChase

Dependent random pointer chase, 16 384 steps, one thread, median of 101 reps.

- **Sattolo cycle.** One random cycle over all elements: no short cycles, nothing for a prefetcher.
- **Dependency defeats overlap.** Load $i+1$ needs the value of load i : pure latency, no MLP.



Latency Benchmark – Random pChase

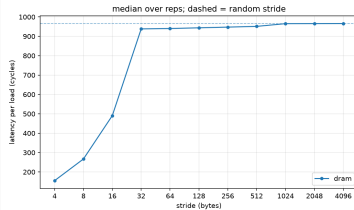
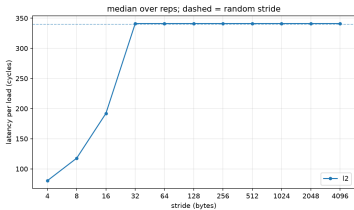
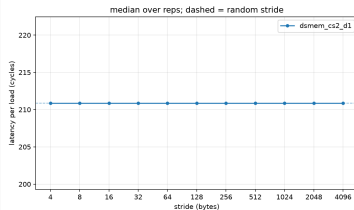
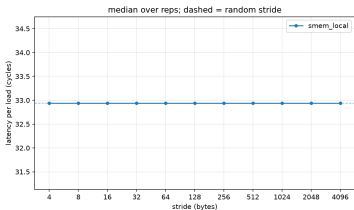
Same chase, but thread 0 of rank 0 accesses a *peer* block's SMEM at rank distance d , median of 101 reps.



- **A genuine SM-to-SM path.** A remote access costs about *half* an L2 hit and a *fifth* of DRAM.
- **Flat across the cluster.** 210.9 cy at distance 1, identically at cluster sizes 2, 4, 8 and 12. Not monotone in distance: ranks pair up $\{2, 3\}, \{4, 5\}, \dots$ — the two SMs of one TPC.
- **Independently confirmed.** H800 agrees within a few cycles; since GB10's L2 and DRAM are *slower* than Hopper's while its fabric is not, DSMEM's advantage is *larger* here.

Latency Benchmark – Linear pChase

Same chase, but element i points to $i + s$, $s = 4\text{ B} \dots 4\text{ KiB}$. Dashed = random chase (previous slide).



- **SMEM/DSMEM** are pattern-blind
- **L2/DRAM** see only the 32 B sector
 $k = 32/s$ loads share one L1 fill,
 $(T + (k - 1) \cdot 42.9)/k$ gives
80/118/192 cy (L2) and
155/266/490 cy (DRAM) — as measured.
- **No hardware prefetcher:** from 32 B on, linear = random.

TODO

- quanto si fa fatica a fare lo scheduling in concorrenza con altri processi (che impatto ha lo scheduler)
- un'altra versione del benchmark in cui non è solo il thread 0 a fare il chase, ma tutti i thread del cluster fanno il chase in concorrenza, e si misura quanto aumenta la latenza per via dello scheduling.
- cosa succede se invece di una random chase usiamo una linear chase, cioè un buffer in cui ogni elemento punta al successivo, e l'ultimo punta al primo. In questo caso il prefetcher dovrebbe essere in grado di anticipare le richieste e ridurre la latenza.
- provare a usare stride-n
- tenersi la varianza, prendi le run, emetti tutti i numeri, passare i parametri da cli, ogni programma una casistica
- slides con disegni, come funziona l'accesso, il warmup
- fare grafici
- usare spatzman riproducibilità, come si fa a garantire che il benchmark sia riproducibile

What's next?

- Throughput benchmark?